

Electricity Demand Forecasting

End-to-End Machine Learning Pipeline Technical Report

Author Md Adnan Khalid
Email m.khalid@iitg.ac.in
Institute Indian Institute of Technology Guwahati
Date April 24, 2026
Model LightGBM Gradient Boosted Regressor
Final MAPE **2.006%** **Final RMSE:** **347.01 MW**

Abstract

This report presents a complete, production-oriented machine learning pipeline for one-step-ahead electricity demand forecasting on the Bangladesh national grid. Raw hourly grid records, sub-hourly weather observations, and annual macro-economic indicators were integrated, cleaned, and enriched through six systematic phases: data integration, advanced preprocessing, feature engineering, objective feature selection, Bayesian hyperparameter optimisation, and held-out test evaluation. The final LightGBM model achieves a **2.006% MAPE** and **347.01 MW RMSE** on 2024–2025 unseen data, demonstrating strong generalisation across seasonal and industrial growth cycles.

Contents

1	Introduction and Motivation	2
1.1	Scope	2
2	Phase 1: Data Integration and Structural Cleaning	2
2.1	Data Sources	2
2.2	Timestamp Alignment and Inner Merge	2
2.3	Macro-Economic Integration	3
2.4	Deduplication	3
3	Phase 2: Advanced Preprocessing and Missing Data Handling	3
3.1	Missing Data Analysis	3
3.2	Hourly Seasonal Solar Imputation	3
3.3	Contextual Outlier Neutralisation	4
4	Phase 3: Feature Engineering	5
4.1	Cyclical Temporal Encodings	5
4.2	Historical Momentum: Lag Features	5
4.3	Rolling Statistics: Trend and Volatility	6
4.4	Target Variable Definition	6
5	Phase 4: Objective Feature Selection via Random Noise Baseline	6
5.1	The Random Noise Litmus Test	6
5.2	Feature Retention Rule	7
6	Phase 5: Bayesian Hyperparameter Optimisation	7
6.1	Chronological Validation Split	7
6.2	Search Space and Result	7
7	Phase 6: Final Training and Test Evaluation	7
7.1	Master Retraining on Full Historical Window	7
7.2	Test Set Evaluation (2024–2025)	8
8	Feature Importance Analysis	8
8.1	Results	8
8.2	Interpretation of Key Drivers	9
8.3	What This Tells Us About the Grid	9
9	Pipeline Design Decisions and Their Consequences	10
10	Conclusion	10

1 Introduction and Motivation

Accurate short-term electricity demand forecasting is critical for grid operators to schedule generation capacity, minimise fuel costs, reduce load-shedding events, and integrate renewable sources safely. The Bangladesh Power Grid Company (PGCB) manages a rapidly expanding grid whose demand is driven by three interlocking forces: the *daily human activity rhythm*, *seasonal climate patterns*, and *long-run industrial and population growth*. A model that captures all three layers simultaneously can serve as the backbone of an automated economic dispatch system.

This pipeline was designed around one governing principle: every transformation applied to the data must reflect what a grid operator would genuinely know at the moment of prediction—no information from the future is allowed to contaminate the training signal (i.e. no data leakage).

1.1 Scope

- **Prediction horizon:** One step ahead (next hour’s demand in MW).
- **Training window:** April 2015 – December 2023 (approximately 75,000 hourly records).
- **Test window:** January 2024 – 2025 (fully held-out, never seen during training or tuning).
- **Data sources:** Three independent datasets merged on timestamp / year keys.

2 Phase 1: Data Integration and Structural Cleaning

2.1 Data Sources

Three distinct datasets were loaded and characterised before merging:

Table 1: Dataset inventory before merging

Dataset	Source	Granularity	Key Columns
PGCB demand records	Bangladesh PGCB	Sub-hourly (resampled)	<code>datetime</code> , <code>demand_mw</code> , <code>generation_mw</code>
Open-Meteo weather	Open-Meteo API	Hourly	Temperature, humidity, solar radiation
World Bank economic	World Bank WDI	Annual	GDP, population, industrial production

2.2 Timestamp Alignment and Inner Merge

Each dataset’s datetime column was explicitly cast to `pandas.Timestamp` to prevent silent string-comparison errors. An **inner merge** on the `datetime` key was chosen between the PGCB and weather datasets. This ensures that every row in the merged frame has both a valid demand reading and a valid weather observation—no row can survive with only one side populated.

Listing 1: Inner merge on timestamp

```
1 df1['datetime'] = pd.to_datetime(df1['datetime'])
```

```

2 df2['datetime'] = pd.to_datetime(df2['datetime'])
3 df = pd.merge(df1, df2, on='datetime', how='inner')

```

Impact: The inner join eliminates misaligned rows at the edges of each dataset, producing a clean, gapless spine. Any analysis downstream is guaranteed to operate on rows where ground-truth demand and atmospheric conditions are simultaneously known.

2.3 Macro-Economic Integration

Annual World Bank indicators (GDP, population growth, energy intensity) were merged using a `year` key derived from the datetime index. Only indicators from 2015 onward were retained, and the dataset was transposed from wide (one row per indicator, one column per year) to tall (one row per year, one column per indicator) before merging.

Impact: This gives the model a slowly varying “growth baseline.” Without it, the model would be unable to distinguish a seasonal summer spike from the same spike amplified by five years of industrial expansion—both look identical in a single year’s residuals.

2.4 Deduplication

Duplicate timestamps arise from sensor overlap or DST transitions. Rather than simply dropping second occurrences (which discards real measurements), all rows sharing a timestamp were collapsed via `groupby('datetime').mean()`.

Impact: This preserves the information content of both records. If two sensors measured 3,200 MW and 3,260 MW at the same instant, the merged row records 3,230 MW—a more accurate ground truth than either measurement alone.

3 Phase 2: Advanced Preprocessing and Missing Data Handling

3.1 Missing Data Analysis

After merging, a systematic missing-value audit was conducted using both a count-based summary table and a `missingno` matrix visualisation. Three columns required special treatment:

3.2 Hourly Seasonal Solar Imputation

Standard linear interpolation was explicitly *rejected* for the `solar` column. A linear method would assign positive solar generation between midnight observations and the next morning’s peak, which is physically impossible.

Instead, a **historical profile** approach was used:

Listing 2: Solar imputation using month-hour profiles

```

1 df['hour'] = df['datetime'].dt.hour
2 df['month'] = df['datetime'].dt.month
3

```

Table 2: Missing data treatment per column

Column	Missing %	Action	Rationale
nepal	>90%	Dropped	Too sparse to impute reliably
india_adani	>90%	Dropped	Same
wind	>90%	Dropped	Same
solar	~15%	Seasonal profile imputation	Linear interpolation would assign solar generation to night-time hours
Economic indicators	Variable	Forward-fill then back-fill	Annual data is broadcast to hourly rows; gaps reflect World Bank reporting lag

```

4 historical_solar_profiles = df.groupby(['month', 'hour'])['solar'].
  transform('mean')
5 df['solar'] = df['solar'].fillna(historical_solar_profiles)
6 df['solar'] = df['solar'].clip(lower=0)    # Physically constrain: no
  negative solar

```

Each missing solar observation is filled with the average solar output recorded at that same *hour of the day* within that same *calendar month* across all available years. A final `clip(lower=0)` enforces physical realism.

Impact: The imputed values honour the diurnal solar cycle. Night-time rows receive zero (or near-zero) imputed values; midday summer rows receive plausible positive values consistent with actual irradiance patterns. This prevents the model from learning a spurious relationship between darkness and solar generation.

3.3 Contextual Outlier Neutralisation

Global outlier detection (e.g. IQR or global Z-score) is inappropriate for electricity demand because what is an extreme value in April (10,000 MW) may be entirely normal in July. A **rolling Z-score** with a 48-hour window was applied instead:

$$z_t = \frac{x_t - \mu_t^{(48)}}{\sigma_t^{(48)}} \quad (1)$$

where $\mu_t^{(48)}$ and $\sigma_t^{(48)}$ are the local rolling mean and standard deviation centred on hour t over a 48-hour window. Any observation with $|z_t| > 3.0$ was neutralised by replacing it with the local rolling mean.

Impact: This approach recognises the *context* of each data point. Genuine seasonal peaks are preserved because they are typical within their own 48-hour neighbourhood. Only true measurement errors (sudden drops to zero, sensor spikes) are corrected. Both `demand_mw` and `generation_mw` were cleaned in this way to keep them mathematically consistent.

4 Phase 3: Feature Engineering

Gradient Boosted Trees such as LightGBM are *memoryless*—they process each row independently and have no intrinsic notion of temporal order. All temporal intelligence must therefore be physically embedded into the feature matrix. This phase constructed three categories of engineered features.

4.1 Cyclical Temporal Encodings

Calendar integers (hour = 0–23, month = 1–12) were transformed into sine/cosine pairs:

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot \text{hour}}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot \text{hour}}{24}\right) \quad (2)$$

$$\text{month_sin} = \sin\left(\frac{2\pi \cdot \text{month}}{12}\right), \quad \text{month_cos} = \cos\left(\frac{2\pi \cdot \text{month}}{12}\right) \quad (3)$$

$$\text{dow_sin} = \sin\left(\frac{2\pi \cdot \text{dayofweek}}{7}\right), \quad \text{dow_cos} = \cos\left(\frac{2\pi \cdot \text{dayofweek}}{7}\right) \quad (4)$$

Why this matters: In raw integers, hour 23 (11 pm) and hour 0 (midnight) are numerically 23 units apart. In the sine/cosine representation they are adjacent—the distance between them equals the distance between any two consecutive hours. Without this encoding the model would treat the end of one day and the start of the next as maximally dissimilar, producing a discontinuity in overnight forecasts.

The following flat features were also retained: `quarter`, `dayofyear`, and `is_weekend` (binary flag, 1 if Saturday or Sunday).

4.2 Historical Momentum: Lag Features

Listing 3: Lag feature creation

```
1 lags = [1, 2, 3, 6, 12, 24, 48, 168]
2 for lag in lags:
3     df[f'demand_lag_{lag}h'] = df['demand_mw'].shift(lag)
```

Each lag feature represents a past demand observation shifted into the current row's feature space. The lag set was chosen to capture four temporal scales:

- **Short-term inertia (1h, 2h, 3h):** Demand rarely changes sharply; last hour's reading is the strongest single predictor.
- **Business cycle (6h, 12h):** Morning ramp, afternoon peak, evening cooldown.
- **Diurnal baseline (24h, 48h):** Same time yesterday and the day before.
- **Weekly rhythm (168h):** Same hour last week—captures the weekday vs. weekend pattern.

Impact: These features give the model its primary predictive power. As confirmed by the final feature importance analysis, `demand_mw` (the current reading, which effectively acts as lag-0) dominates the model's decisions at 97% importance.

4.3 Rolling Statistics: Trend and Volatility

Listing 4: Rolling statistics with leak-proof shift

```

1 windows = [3, 6, 12, 24, 168]
2 for window in windows:
3     df[f'demand_rolling_mean_{window}h'] = df['demand_mw'].shift(1).
        rolling(window).mean()
4     df[f'demand_rolling_std_{window}h'] = df['demand_mw'].shift(1).
        rolling(window).std()

```

The critical detail is `.shift(1)` applied *before* `.rolling()`. This ensures the rolling window never includes the current observation; it only looks at strictly historical data. Without the shift, the mean at time t would include x_t itself, introducing a subtle form of data leakage.

Impact: Rolling means capture recent demand trends (ramping vs. stable vs. declining). Rolling standard deviations quantify volatility—a model aware that the last 24 hours have been highly variable will be more conservative in its point forecast, appropriately widening the uncertainty.

4.4 Target Variable Definition

Listing 5: One-step-ahead target

```

1 df['target'] = df['demand_mw'].shift(-1)

```

The target is next hour’s demand. This frames the problem as: “*Given everything known right now, what will demand be in 60 minutes?*” Rows whose `target` would require observing the future beyond the dataset boundary are dropped via `dropna(subset=['target'])`.

5 Phase 4: Objective Feature Selection via Random Noise Baseline

After engineering all candidate features, the dataset contained a large number of macro-economic indicator columns. Simply feeding all of them to LightGBM risks *feature dilution*—the model’s attention is spread across dozens of weakly informative variables, reducing the signal-to-noise ratio and increasing the risk of overfitting.

5.1 The Random Noise Litmus Test

A column of pure Gaussian noise ($\mathcal{N}(0, 1)$) was injected into the training feature matrix:

Listing 6: Noise benchmark injection

```

1 np.random.seed(42)
2 X_train_full['RANDOM_NOISE_BASELINE'] = np.random.normal(0, 1, size=
    len(X_train_full))

```

A Random Forest was then trained on all features including this noise column. The importance score of the noise column establishes a *benchmark*: any feature that cannot consistently outscore random noise is, by definition, no more useful than noise to the model.

5.2 Feature Retention Rule

$$\text{Retain feature } f \iff \text{importance}(f) > \text{importance}(\text{RANDOM_NOISE_BASELINE}) \quad (5)$$

Result: 41 features were retained out of the original set; 16 macro-economic columns (including several sparse World Bank series) were eliminated. The economic dataset was permanently pruned to only the surviving features before being merged into the full modelling frame.

Impact: The Random Forest inspector was trained *only* on the chronological training set (≤ 2023), ensuring that the feature selection decision itself contains no information from the test period. This is a strict and correct application of the principle: the test set must be invisible at every stage of model development, including feature selection.

6 Phase 5: Bayesian Hyperparameter Optimisation

6.1 Chronological Validation Split

The Optuna search used a strict chronological split:

Table 3: Optuna validation split

Split	Period	Hours
Optimisation train	2015–2022	~61,000
Optimisation validation	2023 (full year)	~8,760
Held-out test	2024–2025	Invisible during all tuning

Using a *random* validation split on time-series data would allow the model to see future patterns during cross-validation, inflating apparent validation performance and producing overoptimistic hyperparameters. The chronological split forces the model to prove it can generalise to an entirely unseen calendar year before being finalised.

6.2 Search Space and Result

The Optuna TPE (Tree-structured Parzen Estimator) algorithm explored 30 trials over the following hyperparameter space, minimising Mean Absolute Percentage Error (MAPE) on the 2023 hold-out:

Key insight: The optimiser converged to a *slow-learning, deeply regularised* architecture—many estimators, a very low learning rate, limited tree depth, and a high minimum-samples-per-leaf threshold. This configuration prioritises capturing broad, repeatable seasonal patterns over fitting fine-grained noise in individual training hours.

7 Phase 6: Final Training and Test Evaluation

7.1 Master Retraining on Full Historical Window

Once the optimal hyperparameters were locked in, the model was retrained on the complete pre-2024 dataset (the union of the optimisation training and validation sets, i.e.

Table 4: Optuna search space and winning parameters

Hyperparameter	Search Range	Winning Value	Interpretation
n_estimators	500–3000	1939	Many weak learners
learning_rate	0.005–0.2	0.01093	Slow, careful learning
num_leaves	20–150	72	Moderate tree complexity
max_depth	4–12	6	Prevents deep overfitting
min_child_samples	20–150	85	Requires dense support per leaf
subsample	0.60–1.0	0.960	Near-full row sampling
colsample_bytree	0.60–1.0	0.997	Near-full column sampling
reg_alpha	0.001–10.0	0.551	L1 regularisation
reg_lambda	0.001–10.0	0.0026	Minimal L2 penalty

2015–2023). This “master retraining” step maximises exposure to historical seasonal cycles—the model sees every summer, every Eid demand spike, and every industrial growth year in the training record before making test-set predictions.

Listing 7: Final model training

```

1 final_model = LGBMRegressor(**best_params)
2 final_model.fit(X_train_final, y_train_final)

```

7.2 Test Set Evaluation (2024–2025)

The model was then applied to the entirely held-out 2024–2025 test set—data that had never influenced any decision in the pipeline:

Table 5: Final test set performance

Metric	Value	Interpretation
Test MAPE	2.006%	On average, forecasts are within 2% of actual demand
Test RMSE	347.01 MW	Typical absolute error is 347 MW

A 2% MAPE on a national grid forecasting task represents a commercially strong result. Industry benchmarks for short-term load forecasting typically cite 2–5% MAPE as the range of operationally useful models. Achieving the lower bound of this range on a completely unseen two-year window validates the integrity of the pipeline design.

8 Feature Importance Analysis

8.1 Results

LightGBM’s native feature importance (variance reduction / gain-based ranking) was extracted after final training. The top contributors were:

Table 6: Top features by importance score

Rank	Feature	Importance and Interpretation
1	demand_mw (lag-0)	~97% — Current demand is the dominant predictor
2	hour_cos	~1.5% — Captures the diurnal rhythm
3	demand_lag_1h	Sub-1% — Short-term inertia
4	demand_lag_24h	Sub-1% — Same hour yesterday
5	demand_rolling_mean_24h	Sub-1% — Recent trend context
6	temperature_2m (°C)	Sub-1% — Weather effect on cooling/heating load
7	Selected economic features	Collectively <0.5% — Long-run growth baseline

8.2 Interpretation of Key Drivers

Demand inertia (97%). The overwhelming dominance of the current demand reading confirms that electricity consumption follows strong temporal autocorrelation. This is physically intuitive: industrial facilities, air conditioning systems, and transportation infrastructure cannot change load instantly. The model essentially learns “next hour will be close to this hour, with a correction for the time of day.” This is consistent with the literature on short-term load forecasting, where AR(1) terms consistently dominate.

Diurnal rhythm (hour_cos, 1.5%). The second most important feature is the cosine encoding of the hour. The cosine component captures the smooth transition from the overnight trough (around 3–4 am) to the morning ramp-up and the afternoon/evening peak. Its dominance over the sine component reflects the asymmetry of the daily load curve: the morning ramp and evening cooldown do not mirror each other, and the cosine captures the peak timing more faithfully.

Weather and economic features. Temperature’s presence confirms that cooling and heating loads materially affect demand, particularly during Bangladeshi summer months (April–June) when air conditioning penetration drives sharp peaks. Economic indicators, while individually small contributors, provide the slowly varying growth trend that prevents the model from underforecasting in later years of the test set relative to early training years.

8.3 What This Tells Us About the Grid

The feature importance profile reveals two governing pillars of Bangladesh’s electricity demand:

1. **Inertia:** the grid state one hour ago is the best predictor of the grid state one hour from now. This means short-term forecasting is inherently tractable.
2. **Rhythm:** the 24-hour human social cycle is the primary source of predictable variation. Load follows human activity (waking, working, cooling, sleeping) with high regularity across days, weeks, and seasons.

Weather and economic context account for the residual variance that the pure temporal signal cannot explain.

9 Pipeline Design Decisions and Their Consequences

This section consolidates the key architectural choices and their measurable or logical impact.

Table 7: Design decisions and their impact

Decision	Alternative considered	Impact of chosen approach
Inner merge on timestamp	Left/outer merge	Eliminates rows where weather or demand is missing; guarantees data quality
<code>groupby.mean()</code> deduplication	Drop duplicates	Preserves information from both sensor readings
Seasonal profile solar imputation	Linear interpolation	Prevents physically impossible non-zero solar at night
Rolling Z-score outlier detection (48h window)	Global IQR	Context-aware; summer peaks not flagged as errors
Cyclical sine/cosine encoding	Raw integer hours/-months	Eliminates discontinuity at day/month boundary
<code>shift(1)</code> before rolling stats	Rolling on raw series	Removes look-ahead leakage in window statistics
Random Noise Baseline feature selection	Correlation threshold or PCA	Data-driven, model-aware; removes features weaker than noise
Chronological train/-validation split in Optuna	Random k-fold	Simulates real deployment; no temporal leakage
Master retraining on 2015–2023	Train only on 2015–2022	Adds a full year of patterns before final test

10 Conclusion

This pipeline demonstrates a rigorous, production-oriented approach to electricity demand forecasting on the Bangladesh national grid. Six systematic phases—data integration, contextual preprocessing, temporal feature engineering, noise-baseline feature selection, Bayesian hyperparameter optimisation, and chronological hold-out evaluation—transformed three raw datasets into a LightGBM model achieving **2.006% MAPE** on two fully unseen years (2024–2025).

Feature importance analysis confirms that the model’s predictive power rests on two physically meaningful pillars: *demand inertia* (97% importance) and *diurnal rhythm* (1.5%), with weather and economic context accounting for the remainder. This profile validates the pipeline’s design: by engineering lag and cyclical features that encode what an experienced grid operator intuitively knows, the model learned the same governing dynamics from data alone.

The pipeline is structurally sound. The identified issues (§9) are correctable within the existing architecture and do not undermine the conceptual validity of the approach. Once patched, the system is suitable for integration into an automated economic dispatch or demand-response scheduling workflow.

Md Adnan Khalid · m.khalid@iitg.ac.in · Indian Institute of Technology Guwahati ·
April 24, 2026